

UNIVERSITY OF COLORADO - BOULDER
Robotics Program

ROBO 5000 (CSCI 5202) - Intro to Robotics

Homework #2 (Assigned: 09/20, Due: 10/04 11:59pm on *Canvas*)
Motion Planning

Instructions

This homework assignment has coding and written components. For written components, please add comments to the end of your file (no pdf submission required). Be sure to comment code so the grading staff can easily follow your logic. **You are encouraged to discuss your work with other students, but submitted work must be your own. Please indicate in the header of each file who your collaborators for this homework are.**

Search-based algorithm preliminaries

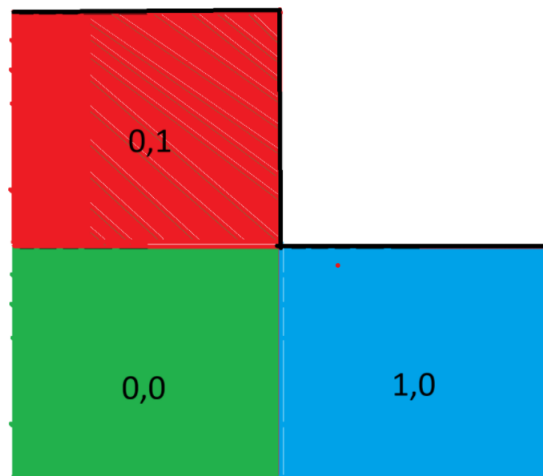
Obstacles. Obstacles are drawn as black dots, but each dot is only an *endpoint*. Every pair of dots is joined by a solid straight line, and that line is the obstacle (i.e., horizontally from the leftmost dot to the rightmost dot, and vertically from the bottom dot to the top dot). No path may cross such a line at any point along it.

For example, in the Question 1 figure the dots at $(0,0)$ and $(60,0)$ are joined into a single horizontal wall spanning everything between them, so there is no free space anywhere along it.

From continuous space to a grid. The workspace is divided into squares measuring 2 units $\times$ 2 units, and each square becomes one node of the graph. A point landing exactly on a shared edge is resolved by one rule:

Every square owns its left and bottom edges, but not its top or right edges.

So, a point on a shared edge always belongs to the square above it or the square to its right.

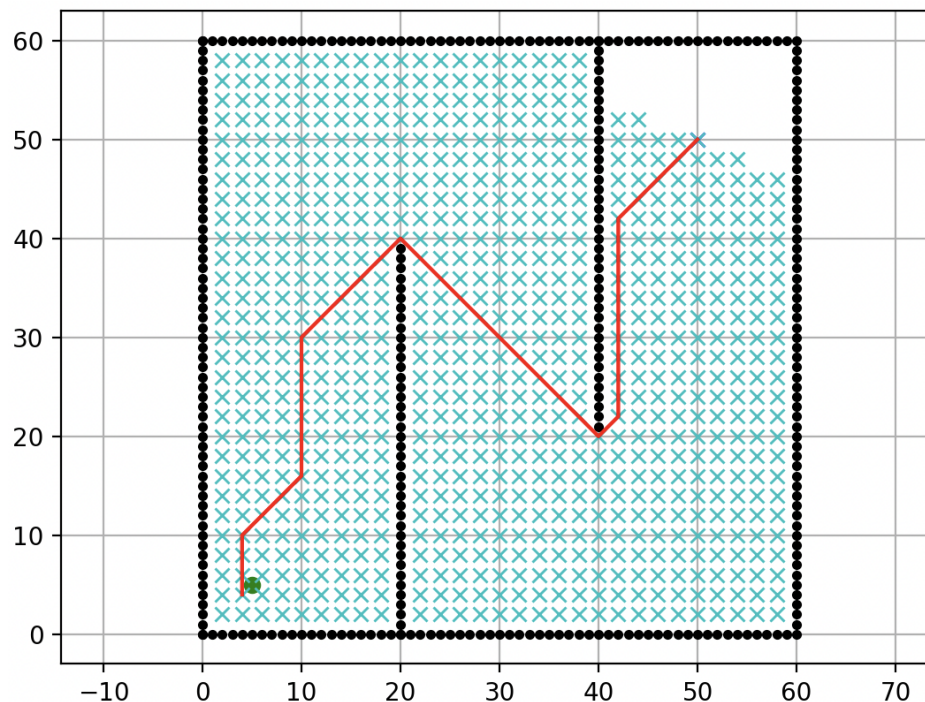


The figure shows what this means for the green square, $(0,0)$:

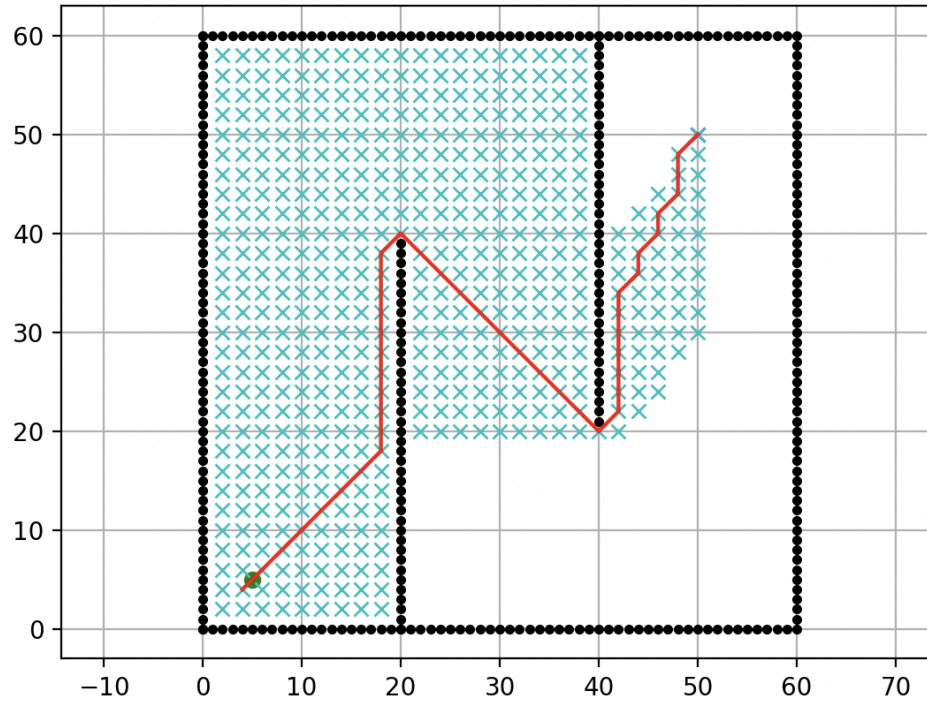
- a point on its **bottom or left** edge stays in the green square, $(0, 0)$;
 - a point on its **top** edge moves up to the red square, $(0, 1)$, because that edge is the red square's bottom;
 - a point on its **right** edge moves right to the blue square, $(1, 0)$, because that edge is the blue square's left;
 - a point on its **top-right corner** moves diagonally, to $(1, 1)$.
0. Create a new branch for this homework (a new branch will be used for each assignment in this course). Name your branch **hw2_FIRST_LAST** where you replace **FIRST_LAST** with your first and last name. For example, my new branch would be called **hw2_Caleb_Escobedo**.

Create a directory within your main repo folder named **HW2**. This will serve as the directory where you store all file(s) for this homework. The grading team should be able to run your files from this new directory. Your final submission will simply be a link to this branch.

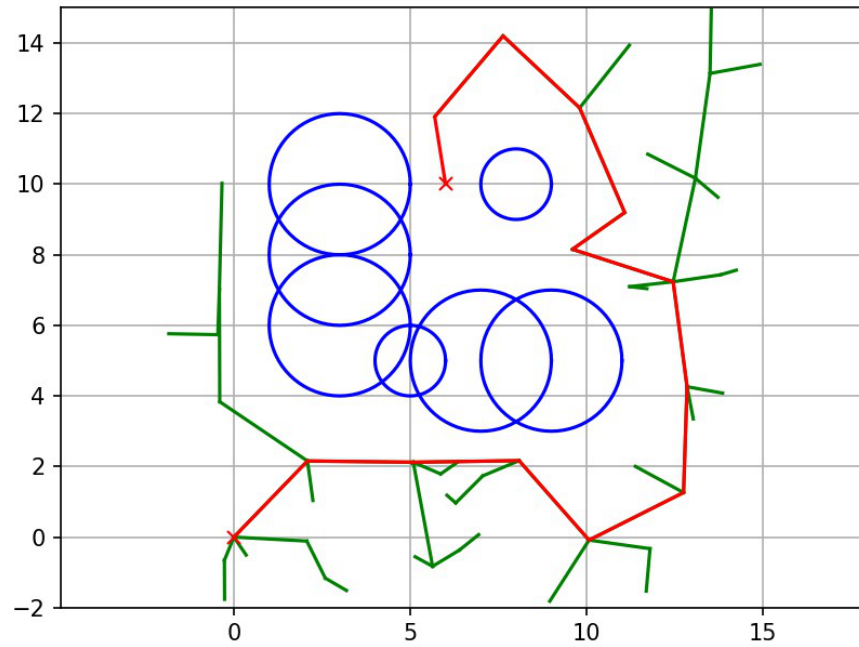
1. [10 points] A template has been provided to implement and visualize Dijkstra's algorithm via `dijkstra.py`. A brief pseudocode has been commented in the python file to help in the implementation. Finish the implementation such that the output is obtained. You are only required to submit your `.py` file as a deliverable (no screenshot/pdf necessary).



2. [10 points] A template has been provided to implement and visualize A* algorithm via `a_star.py`. A brief pseudocode has been commented in the python file to help in the implementation. Finish the implementation such that the output is obtained. You are only required to submit your `.py` file as a deliverable (no screenshot/pdf necessary).



3. [10 points] Change the environments for the A* and Dijkstra's implementation such that the number of nodes explored for each algorithm is the same (within 5%). The A* heuristic function may not be altered to achieve this. The start and goal nodes may not be altered either - only the location of the obstacle points may change. The addition of the number of nodes in the OPEN and CLOSED sets should be used as a measure of the number of nodes explored when comparing the algorithms. Please make sure that your updated environment is marked clearly in your code so that we can test it.
4. [2 points] Assuming an optimistic heuristic, is it ever possible for A* to be slower (in terms of time) than Dijkstra's at finding a solution? If so, give an example.
5. [10 points] A template has been provided to implement and visualize RRT algorithm via `rrt.py`. A brief pseudocode has been commented in the python file to help in the implementation. Finish the implementation such that a similar output is obtained (Since RRT is probabilistic, the exact output isn't expected. However, the path between the start and goal must be found). **Note:** you may assume that your agent is completely holonomic.



6. [3 points] Does the above implementation work if the value of `expand_dis` is changed from 3.0 to 0.1? Why or why not? If not, what parameter can be changed to ensure the implementation works with `expand_dis` being 0.1?